

CHAPTER 15

Advanced Terraform Topics

Multi-Cloud Deployments, Custom Providers, Dynamic Data & Performance

Instructor Deck • Terraform Fundamentals Series



What We'll Cover



15.1

Managing Multi-Cloud Deployments

AWS + Azure in one configuration



15.2

Writing Custom Terraform Providers

Extend Terraform to unsupported services



15.3

Using Dynamic Data Sources

Pull live data with the external provider



15.4

Performance Optimization

Parallelism, remote state, module caching

Step 1: Define Providers for AWS & Azure

Modify **providers.tf** to declare both providers — Terraform manages both clouds from one config.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

provider "azurerm" {
  features {}
}
```



Two providers, one state

Terraform's provider system lets a single configuration coordinate resources across completely different clouds.

- Each provider block configures its own credentials/region
- Resource types are prefixed by provider (aws_, azurerm_)
- State tracks both clouds' resources together

Step 2: Deploy Across Both Clouds

EC2 instance (AWS)

```
resource "aws_instance" "web" {  
  ami          = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
}
```

Virtual Machine (Azure)

```
resource "azurerm_virtual_machine" "vm" {  
  name      = "azure-vm"  
  location  = "East US"  
  vm_size   = "Standard_B1s"  
  ...  
}
```



terraform apply

provisions both resources in a single run



One Terraform run, resources live in two clouds simultaneously.

`aws_instance.web`

`azurerm_virtual_machine.vm`

Extending Terraform with Your Own Provider

When a service isn't officially supported, build a custom provider using the Terraform Plugin SDK.

1

Install the Plugin SDK

```
go install github.com/hashicorp/terraform-plugin-sdk/v2
```

Step 2: Create a Custom Provider in Go — example fetches weather data

```
package main

import (
    "github.com/hashicorp/terraform-plugin-sdk/v2/helper/schema"
)

func Provider() *schema.Provider {
    return &schema.Provider{
        ResourcesMap: map[string]*schema.Resource{
            "weather_temperature": resourceWeatherTemperature(),
        },
    },
}
```



Result

This registers a new resource type, `weather_temperature`, that Terraform can create, read, update, and destroy — just like any built-in resource.

Pulling External Data with the external Provider

Terraform HCL — calls a script and captures its output

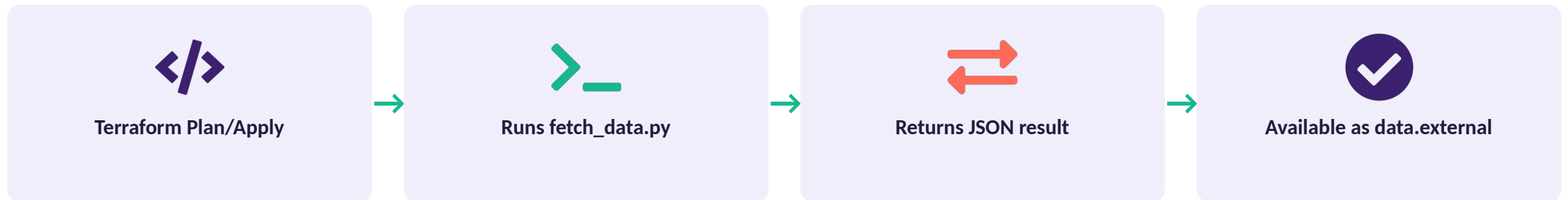
```
data "external" "example" {
  program = ["python3",
    "${path.module}/fetch_data.py"]
}

output "api_data" {
  value = data.external.example.result
}
```

fetch_data.py — returns JSON to stdout

```
import json

print(json.dumps({
  "result": "Hello from API!"
}))
```



Making Terraform Runs Faster



Parallel Execution

Run independent resource operations concurrently for faster apply/destroy cycles.



Remote State + Locking

Use S3 + DynamoDB (or similar) so teams share state safely without corruption.



Module Caching

Cache frequently used modules locally to avoid redundant downloads on every run.

```
$ terraform apply -parallelism=10 # faster deployments via concurrent runs
```

SCENARIO

Deploy Infrastructure Across AWS & Azure



Problem: A company needs infrastructure across AWS and Azure, managed using one Terraform configuration.

Solution



Expected Outcome

- ✓ Terraform manages resources across multiple clouds
- ✓ Dynamic API data is fetched in real-time
- ✓ Optimized Terraform execution for faster deployments

What We Covered



Deploying infrastructure across multiple clouds



Building a custom Terraform provider



Using dynamic data sources to pull external API data



Optimizing Terraform performance